SentinelLoop — Self-Healing Machines (Case Study 1)

Team SentinelLoop · (college to be filled)

Thesis

Closed-loop prognostics: RUL with a calibrated 90% interval; a deterministic Triage Agent raises work orders from the interval lower bound, escalates untrustworthy inputs to a human, retrains only through a gate, and keeps every decision auditable. No LLM · open-source Python · all four C-MAPSS subsets · one command.

Problem

PROBLEM (in the case study's words): a fleet streams sensor data; today failures are found after breakdowns or by fixed service intervals.

Needed: ingest → clean → detect abnormal behaviour → predict Remaining Useful Life → create the work order automatically → involve a human only for except

Ground rules we design to: mild imbalance (failures are rare), no manual steps once data is provided, lightweight rules (no LLM), documented design.

Architecture: data → features → model → agent → work order

C-MAPSS SENSOR STREAM: 21 sensors + 3 settings batch file or cycle-by-cycle replay

CLEAN: per-unit fill + quality flag drop list · operating-regime scaling (fit on train only)

FEATURES: causal rolling mean / std / slope over 20 cycles (never look ahead)

MODELS: health index on a healthy baseline HistGradientBoosting RUL + conformal 90% interval

TRIAGE AGENT (rules): severity from the interval LOWER bound; quality, drift and width checks → escalate to a human

WORK ORDER: one open order per machine action · due-by cycle · sensors audit chain · SQLite / JSON

Side loop: drift → retrain → gate → registry

DRIFT MONITOR: age-matched PSI per feature settings = critical columns

RETRAIN ON NEW DATA: `sentinelloop retrain`: new file or drift alarm → challenger

PROMOTION GATE: validation RMSE ≤ +2% · coverage ≥ 0.85 · late orders not worse

MLflow REGISTRY: SQLite backend · champion alias reject logged with reasons

the retraining flow shows both outcomes: a promoted challenger and a rejected one, each with its reasons

Data engineering and ML approach

Data engineering approach

Every fit on training engines only: constant-sensor drop list,
KMeans operating regimes (6 on FD002/FD004, purity 100%),
per-regime scaling of every sensor.
Missing values: per-unit fill + a quality flag the agent reads.
Noise: rolling-window features (EMA available; hurt FD004, so off).
Causal windows: same arithmetic in batch and stream (tested bitwise).
Honest validation: engine-level folds with random truncation.
The leakage-prone 'last cycle of a held-out engine' protocol
reports RMSE 2.5–6.9; the honest one 12.4–16.6.
We report the honest one, against capped AND raw truth.
Every artefact carries the data SHA-256, config hash, MLflow run id.

ML approach and reasoning

Anomaly: healthy-baseline health index (RMS of the two largest
standardised deviations from each engine's early life); names
the sensors; armed after the window fills; 'mild imbalance'.
RUL: HistGradientBoosting (CPU, seconds, retrains live) with
conformalized quantile regression intervals calibrated on
held-out engines (Romano et al. 2019; CQR on C-MAPSS as in
Robinson, IJPHM 2026; cited, not claimed). Coverage is measured
per subset and stated as marginal, never as a guarantee.
Why not an LSTM: published deep models reach ≈ 11–13 RMSE on

Triage Agent logic and MLOps

Team SentinelLoop · (college to be filled)

Rule table (deterministic, on the 90% interval lower bound L)

Condition (L = interval lower bound)	Severity	Action
L ≤ 10 cycles	CRITICAL	stop & repair, order
L ≤ 25	HIGH	maintenance order
L ≤ 50	MEDIUM	plan maintenance
sustained anomaly, RUL healthy	WATCH	monitor
interval wider than calibrated limit, > 10% imputed, setting out of range, fleet drift on HIGH/CRITICAL, conflict	ESCALATE	inspection, human

Work orders, audit, retraining

One open order per machine; a higher severity upgrades it and pulls its due-by cycle earlier. Every decision stores inputs, thresholds, rules fired and model version in a hash-chained audit log that re-derives 100% of decisions. Mild imbalance: the anomaly baseline is healthy behaviour; thresholds are set on validation engines, never on test. MLOps: MLflow (SQLite) tracks params, metrics, data hash, versions; `retrain` trains a challenger on new data, gates it, promotes or rejects it with the reasons logged. One command runs everything: python -m sentinelloop run --subset all

Dataset coverage, measured status, next steps

Team SentinelLoop · (college to be filled)

Coverage and measured results (official test protocol)

All four subsets through configuration (6 regimes on FD002/FD004; two fault modes on FD003/FD004).

Already running on a CPU laptop, official last-cycle test protocol, RMSE vs RUL capped at 125 / raw:

FD001 14.2 / 15.4 · FD002 14.9 / 27.0 · FD003 13.6 / 15.3 · FD004 15.1 / 26.9

90% interval coverage 0.94 / 0.89 / 0.92 / 0.85 (FD004 below nominal, reported) · naive mean 41.9–45.6 · RandomForest 14.5–16.2 · PHM08 score with it

Tools and Round 2

Tools: Python 3.12 · pandas · scikit-learn · MLflow · Streamlit · pytest suite. Dataset: NASA C-MAPSS (Saxena et al., PHM08; archive verified by checksum, not redistribu

Round 2 adds: policy-replay evaluation of the agent, stress tests on corrupted inputs, the live retraining flow (promote and reject), the monitoring dashboard, a 10-slid